

# HARD INSTANCES OF DISCRETE LOGARITHM PROBLEM AND CRYPTOGRAPHIC APPLICATIONS

CHRISTOPHER BATTARBEE, ARMAN DARBINYAN, DELARAM KAHROBAEI

ABSTRACT. Let  $f : \mathbb{N} \rightarrow \mathbb{N}$  be an arbitrary integer valued function. The goal of this note is to show that one can construct a finitely generated group in which the discrete log problem is polynomially equivalent to computing the function  $f$ . In particular, we provide infinite, but finitely generated groups, in which the discrete logarithm problem is arbitrarily hard. As a corollary, we construct a family of two-generated groups that have polynomial time word problem and NP-hard discrete log problem. Additionally, using our framework, we propose a generic scheme of cryptographic protocols, which might be of independent interest.

## 1. INTRODUCTION

The Discrete Logarithm Problem (DLP) is the foundational problem of public-key cryptography. Its security underpins that of the Diffie-Hellman key exchange, which is, for example, the mechanism for establishing secrets in TLS 1.3. Despite its instrumental role in facilitating billions of everyday secure communications, the difficulty of solving the DLP remains a subject about which relatively little is known.

The classical result providing a lower bound on the complexity of DLP is that of Shoup, presented in [Sho97]: in something called the “generic group model”, one can show that an algorithm solving DLP in a generic group  $G$  must perform  $\mathcal{O}(\sqrt{p})$  operations, where  $p$  is the largest prime divisor of  $|G|$ . Since many of the best-known algorithms for DLP have about this complexity we can conclude that they are more or less optimal.

Shoup’s result requires treating groups in a very generic way, and one’s mileage may vary if more information about the group elements is revealed. This is of particular interest for DLP on elliptic curves, which is the flavour of the problem most commonly encountered in modern communication. On the other hand, as far as we are aware, there are no prior results whereby more information about the group actually *improves* the Shoup bound. We accomplish this in this work.

Indeed, in this direction, an obvious generalization of the problem to study is DLP in an infinite group. Very little study exists on this subject - in this paper, we demonstrate that by

---

*Date:* November 26, 2025.

*Key words and phrases.* Discrete Logarithm Problem, Post-quantum Cryptography, Infinite Groups, NP-hardness.

moving to the infinite context, it is not only possible to obtain plausibly difficult instances of DLP, but instances of DLP that are as difficult as the evaluation of any function on the integers.

More precisely, this paper provides a translation of the theory of one-way functions to an infinite-group-theoretic setting. We show that for any function  $f : \mathbb{N} \rightarrow \mathbb{N}$ , there exists an infinite abelian group in which the word problem is solvable in about the same time as the evaluation of  $f^{-1}$ , but the discrete logarithm problem is at least as hard as the evaluation of  $f$ . Loosely speaking, the ease of the word problem corresponds to the ability to easily compute group operations, so indeed, if  $f$  is easy to invert and hard to compute, we have a tractable object in which DLP is as hard as desired. Up to the existence of one way functions, we generate controllably hard instances of DLP.

Let  $f : \mathbb{N} \rightarrow \mathbb{N}$  be a computable function. Let us denote by  $T_f : \mathbb{N} \rightarrow \mathbb{N}$  the maximal number of steps that it takes to compute  $f(m)$  for  $1 \leq m \leq \exp(n)$  on a fixed Turing machine that computes  $f$ . In other words,  $T_f$  corresponds to the time complexity of computing  $f$ . Similarly, define  $T_{f^{-1}} : \mathbb{N} \rightarrow \mathbb{N}$  as  $T_{f^{-1}}(n)$  is the maximum number of steps needed to verify whether or not  $f(k) = l$  for any input  $(k, l) \in \mathbb{N} \times \mathbb{N}$  satisfying  $1 \leq \exp(kl) \leq n$  on a fixed Turing machine that solves the verification problem. In other words,  $T_{f^{-1}}$  corresponds to the time complexity of the problem of verification of the correctness of the computation of  $f$ . We also say that an algorithmic problem  $P_1$  is polynomial time equivalent to the problem  $P_2$  if the time complexity for  $P_1$  differs from the time complexity of  $P_2$  at most by a polynomial factor.

Our main results obtained in this work are the following.

**Theorem 1.1** (See Section 3). *For any computable function  $f : \mathbb{N} \rightarrow \mathbb{N}$ , there exists a two-generator group  $G_f$  such that the discrete log problem in  $G_f$  can be solved in time  $T_f$  and the word problem can be solved in time  $T_{f^{-1}}$ , both up to polynomial factors.*

The next theorem can be regarded as a corollary from Theorem 1.1.

**Theorem 1.2** (Corollary 3.6). *There exist two-generated groups with polynomial time word problem and NP-hard discrete log problem.*

**1.1. Related work.** For a survey on the classical difficulty of solving DLP in a generic group, in the multiplicative group of a finite field, and in an elliptic curve, we recommend the survey article [JOP14].

There are some papers describing concrete instances of groups in which the generic lower bound is violated (or at least, make progress towards demonstrating this); that is, extra information about the group allows one to give an algorithm *upper-bounded* asymptotically below the Shoup

bound. Important examples of this type of work include [LO91, BGJT14, MVO91]. Nevertheless, as far as we are aware, there are no examples in the literature of instances of concrete groups in which the lower bound for DLP is improved compared to the Shoup bound.

**1.2. The main premise behind the group theoretical constructions in our paper.** In the current paper we present several cryptographic protocols implemented in a setting of finitely generated infinite groups. To achieve this implementation, we start with (abelian) groups of countable rank, then describe constructions that allow us to carry essential algorithmic and cryptographic information that presentations of those abelian groups carry into the setting of finitely generated groups. The main conceptual premise is that for groups that are not finitely generated, most of the natural algorithmic properties are not intrinsic for groups themselves but intrinsic for particular enumerated presentations (i.e. enumerations of their elements), whereas for finitely generated groups those properties can be regarded as intrinsic properties of the groups themselves. An example of such an algorithmic property is the word problem in groups.

**1.3. Quantum resistance.** In the current landscape of the first standardised primitives believed to be quantum-resistant, a natural question to ask is the following: how resistant to quantum attack do we expect the DLP to be in the groups we have described? This is especially pertinent in light of the high-profile cryptanalytic surprises that emerged on the road to standardisation: in particular the recent attacks [Beu22, TPD21, BBC<sup>+</sup>21] against the round-3 DSSs Rainbow [Beu22] and GEMSS [CFMR<sup>+</sup>17], as well as the cryptanalysis of round-4 isogeny-based KEM known as SIKE [CD22, MM22]. In other words, despite the completion of the standardisation process, there remains motivation to study a variety of candidate quantum-hard problems.

This work does not address the quantum resistance of DLP in the groups we introduce. On the one hand, the group we work in is somehow defined with respect to a group whose elements all have finite order, and so we would expect some Shor-like method period-finding to apply; on the other hand, the actual group  $G_f$  in which the difficulty of DLP is to be analysed has elements of infinite order, and so the standard quantum techniques might not be expected to work. In order to settle the question, one would have to understand how the difficulty of the problem in these two groups is related from a quantum perspective, as well as understanding how quantum algorithms interact with symbolic algebra. These problems are considered rich enough to warrant further work. Indeed, during the preparation of this manuscript, a work giving detailed analysis of the hidden subgroup problem for infinite groups appeared on the arXiv [Kup25], so certainly there is interest in this kind of problem in the community.

## 2. ENUMERATED ABELIAN GROUPS AND ALGORITHMIC PROBLEMS ON THEM

Let

$$(1) \quad A = \langle a_1, a_2, \dots \mid [a_i, a_j], i, j \in \mathbb{N}; R \rangle$$

be an abelian group with an enumerated set of generators  $\{a_1, a_2, \dots\}$ , the set of relators  $[a_i, a_j] := a_i a_j a_i^{-1} a_j^{-1}$  and an additional set of relators denoted by  $R$ . In this work, we describe how algorithmic properties of the presentation (1) can be carried out to a two-generated groups, making those properties intrinsic for the group itself and not just of its presentation. As main applications, we describe a family of finitely generated groups for which the Discrete Log Problem is polynomially equivalent to the computation of any function  $f : \mathbb{N} \rightarrow \mathbb{N}$  fixed beforehand. This is carried out in Sections 3.

Next, in Section 4, we describe how this idea can be implemented in cryptographic setting by describing a general scheme of a key-exchange protocol that encapsulates properties of commutative actions of groups and one-way functions via encodings in two-generated infinite groups.

**2.1. The Discrete Log problem in countable abelian groups.** Let  $f : \mathbb{N} \rightarrow \mathbb{N}$  be a fixed computable map. In this subsection, we are going to describe a presentation of a countable abelian group for which the Discrete Log problem is polynomially equivalent to the computation of  $f(n)$ .

By  $T_f : \mathbb{N} \rightarrow \mathbb{N}$  we denote the time complexity of computing the values  $f : \mathbb{N} \rightarrow \mathbb{N}$  on a fixed Turing machine. That is,  $T_f(n)$  indicates the number of steps the machine takes to output  $f(n)$  after input  $n$ .

Similarly, by  $T_{f^{-1}} : \mathbb{N} \rightarrow \mathbb{N}$  we denote the time complexity on a fixed Turing machine for the following problem: on input the pair  $(n, m) \in \mathbb{N} \times \mathbb{N}$  is given. Verify whether or not  $f(n) = m$ .

Define

$$(2) \quad A_f := \langle a_1, a_2, \dots, b_1, b_2, \dots \mid [a_i, a_j], [a_i, b_j], [b_i, b_j], a_i^{f(i)} = b_i, i, j \in \mathbb{N} \rangle.$$

Note that each element of  $A_f$  can be represented as a finite word of the form

$$w = a_{n_1}^{k_1} \dots a_{n_t}^{k_t} b_{m_1}^{l_1} \dots b_{m_s}^{l_s} \in \{a_1^{\pm 1}, a_2^{\pm 1}, \dots, b_1^{\pm 1}, b_2^{\pm 1}, \dots\}^*.$$

Also, note that each term  $a_{n_i}^{k_i}$  can be encoded as a string of 0's and 1's of length  $\Theta(\log(|n_i k_i|))$ . Similarly, the term  $b_{m_j}^{l_j}$  would require a string of size  $\Theta(\log(|m_j l_j|))$ . Thus, we define the length function

$$\ell : \{a_1^{\pm 1}, a_2^{\pm 1}, \dots, b_1^{\pm 1}, b_2^{\pm 1}, \dots\}^* \rightarrow \mathbb{N} \cup \{0\}$$

as follows:

$$\ell(w) := \sum_{i=1}^t \log(|n_i k_i|) + \sum_{j=1}^s \log(|m_j l_j|).$$

**Definition 2.1** (Reduced word). *We say that  $w = a_{n_1}^{k_1} \dots a_{n_t}^{k_t} \in \{a_1^{\pm 1}, a_2^{\pm 1}, \dots\}^*$  is in a reduced form if  $w$  is the empty word or  $n_1, \dots, n_t$  and  $m_1, \dots, m_s$  are pair-wise not-equal, respectively, and  $k_1, \dots, k_t, l_1, \dots, l_s$  are non-zero integers, and it does not contain terms  $a_n^k$  and  $b_n^l$  such that  $k = -lf(n)$ .*

**Lemma 2.1.** *Any word  $w \in \{a_1^{\pm 1}, a_2^{\pm 1}, \dots, b_1^{\pm 1}, b_2^{\pm 1}, \dots\}^*$  can be transformed into a reduced word  $w'$  that represents the same element in  $A_f$  as  $w$  in time  $\mathcal{O}((l(w))^2 + l(w))T_{f^{-1}}(l(w))$ .*

*Proof.* Indeed, given  $w \in \{a_1^{\pm 1}, a_2^{\pm 1}, \dots, b_1^{\pm 1}, b_2^{\pm 1}, \dots\}^*$ , simply by transposition of terms  $a_i^{\pm 1}, b_i^{\pm 1}$ , first we can transform it into a word  $w' = a_{n_1}^{k_1} \dots a_{n_t}^{k_t} b_{m_1}^{l_1} \dots b_{m_s}^{l_s}$  that would satisfy the condition that  $n_1, \dots, n_t$  and  $m_1, \dots, m_s$  are pair-wise not-equal, respectively, and  $k_1, \dots, k_t, l_1, \dots, l_s$  are non-zero integers. This can be done in time  $\mathcal{O}((l(w))^2)$ .

Therefore, without loss of generality, we can assume that  $w$  satisfies the condition that  $n_1, \dots, n_t$  and  $m_1, \dots, m_s$  are pair-wise not-equal, respectively, and  $k_1, \dots, k_t, l_1, \dots, l_s$  are non-zero integers. In this case, to check if  $w$  is in a reduced form, one can check for each term  $a_{n_i}^{k_i}$ ,  $i = 1, \dots, t$ , if there is a term  $b_{m_j}^{l_j}$ ,  $j = 1, \dots, s$ , such that  $n_i = m_j$  and  $k_i = -m_j f(m_j)$ . This step can be done in time  $\mathcal{O}(l(w))T_{f^{-1}}(l(w))$ . Therefore, the assertion of the lemma follows.  $\square$

**Corollary 2.2.** *The word problem in  $A_f$  is solvable in time  $\mathcal{O}(n^2 T_{f^{-1}}(n))$ , where  $n$  is the length of the input word with respect to the length function  $l$ .*

*Proof.* It is enough to notice that for  $w \in \{a_1^{\pm 1}, a_2^{\pm 1}, \dots, b_1^{\pm 1}, b_2^{\pm 1}, \dots\}^*$  given in a reduced form, it represents the trivial element of  $A_f$  if and only if  $w$  is the empty word. Then, the claim of the corollary follows from Lemma 2.1.  $\square$

**Lemma 2.3.** *The Discrete Log problem in  $A_f$  with respect to the presentation (2) of  $A_f$  is polynomially equivalent to the computation of  $f : \mathbb{N} \rightarrow \mathbb{N}$ .*

*Proof.* Indeed, let us consider an equation  $u^x = v$ , where  $x \in \mathbb{Z}$  is the unknown,  $u, v \in A_f$ . Without loss of generality, assume that  $u$  is given in a reduced form as  $u = a_{n_1}^{k_1} \dots a_{n_t}^{k_t} b_{m_1}^{l_1} \dots b_{m_s}^{l_s} \in \{a_1^{\pm 1}, a_2^{\pm 1}, \dots, b_1^{\pm 1}, b_2^{\pm 1}, \dots\}^*$ . Then, it follows from the definition of  $A_f$  that if a reduced form of  $v$  does not contain a generator  $a_{n_i}$  or  $b_{m_j}$  for  $i = 1, \dots, t, j = 1, \dots, s$ , or it contains a generator different from those, then  $u^x = v$  has no solution. Therefore, without loss of generality, let us assume that  $v$  is given in a reduced form as  $v = a_{n_1}^{k'_1} \dots a_{n_t}^{k'_t} b_{m_1}^{l'_1} \dots b_{m_s}^{l'_s}$ .

By introducing dummy multipliers of the form  $a_n^0$  or  $b_m^0$ , without loss of generality, we can assume  $(n_1, \dots, n_t) = (m_1, \dots, m_s)$  and the exponents of the terms are non-necessarily non-zero, which is the only deviation from the definition of the reduced form.

Under this convention, let  $u = a_{n_1}^{k_1} \dots a_{n_t}^{k_t} b_{n_1}^{l_1} \dots b_{n_t}^{l_t}$  and  $v = a_{n_1}^{k'_1} \dots a_{n_t}^{k'_t} b_{n_1}^{l'_1} \dots b_{n_t}^{l'_t}$ . Then, for each  $i = 1, \dots, t$ ,  $x$  must satisfy the system of equations

$$(3) \quad \begin{cases} \lfloor \frac{k_i x}{f(n_i)} \rfloor + l_i x = l'_i \\ k_i x - \lfloor \frac{k_i x}{f(n_i)} \rfloor f(n_i) = k'_i \end{cases}.$$

Note that for each  $i = 1, \dots, t$ , this system can have at most one solution. Therefore, it is enough to find  $x$  when  $i = 1$  then check if it satisfies the systems of equations for  $i = 2, \dots, t$ . Moreover, this  $x$  can be found by formula  $x = \frac{f(n_1)l'_1 + k'_1}{f(n_1)l_1 + k_1}$ . Therefore, assuming that  $f(n_1), \dots, f(n_t)$  are found, one can find  $x$  or show that such  $x$  does not exist in time  $\mathcal{O}(t \log(f(n_1)l_1 l'_1 k_1 k'_1))$ . This implies that finding  $x$  or showing that  $x$  does not exist can be done in time  $\mathcal{O}((\ell(u) + \ell(v))T_{\ell(u) + \ell(v)})$ .

On the other hand, consider the equation  $a_n^x = b_n$ . The solution of this equation is  $x = f(n)$ . This implies that the Discrete Log Problem in  $A_f$  is at least as difficult as computing  $f$ . This completes the proof.  $\square$

### 3. FINITELY GENERATED GROUPS WITH CONTROLLABLE DISCRETE LOG PROBLEM VIA GROUP EMBEDDINGS

In this section, we are going to present an embedding construction of the abelian group  $A_f$  from the previous section into a two-generated group  $G_f$  such that the Discrete Log Problem for  $G_f$  is polynomially equivalent to the computation of  $f : \mathbb{N} \rightarrow \mathbb{N}$ . The construction is an adaptation of a famous embedding construction of Neumann [Neu60] (see also constructions from [Dar15, Dar20]). Throughout this section, let  $f : \mathbb{N} \rightarrow \mathbb{N}$  be a fixed map as in the previous section, and  $A_f$  is as in Section 2.

**3.1. The general idea.** Let  $G_f = \langle x, y \rangle$  be a two generated group (the index  $f$  does not play a role so far, but it's context will be clear later). Then, for each word from  $\{x^{\pm 1}, y^{\pm 1}\}^*$ , given as  $w := x^{k_0} y^{l_1} x^{k_1} \dots y^{l_t} x^{k_t} y^{l_{t+1}}$ , where  $k_i, l_i \neq 0$  for  $1 \leq i \leq t$ , we define its length  $\ell(w)$ , as

$$\ell(w) = \log(|k_0| + 1) + \log(|l_{t+1}| + 1) + \sum_{i=1}^t \log(|k_i l_i| + 1),$$

where for convenience we define  $\log(0) = 0$ . One can think of  $\ell(w)$  as the length (up to the linear equivalence) of  $w$  when encrypted via 0's and 1's. We refer to  $\ell(w)$  simply as *the length of the word  $w$* .

Let

$$\Psi : A_f \hookrightarrow G_f = \langle x, y \rangle$$

be an *effective* embedding of the group  $A_f$  into a two generated group  $G_f$ , where by effective embedding we mean that there is a *polynomial time computable* map (with respect to the length

of the input)

$$\phi : \mathbb{N} \rightarrow \{x^{\pm 1}, y^{\pm 1}\}^*$$

such that for each  $i \in \mathbb{N}$ , the image  $\phi(i)$  is a word in  $\{x^{\pm 1}, y^{\pm 1}\}^*$  that represents the element  $\Psi(a_i)$  in  $G_f$  and such that the length of  $\phi(i)$  is bounded from above by  $P(\log(i))$  for some fixed polynomial  $P(n)$ . For description of such embeddings, see, for example, [Dar15, Dar20].

Next, let us observe that the Discrete Log Problem in  $G_f$  is at least as difficult as computation of  $f : \mathbb{N} \rightarrow \mathbb{N}$ .

Indeed, let us consider the equation  $\Psi(a_i)^x = \Psi(b_i)$  for  $i \in \mathbb{N}$ . Then, its solution is  $x = f(i)$ . Therefore, since  $\Psi$  is an effective embedding, solving it is at least as difficult as computing  $f(i)$ .

In the next subsection we advance this insight by describing a concrete effective embedding of  $A_f$  into a specific two generated group  $G_f$ .

**3.2. An effective embedding of  $A_f$  into  $G_f$ .** We start this section by fixing some standard group theoretical notations. Namely, for a group  $G$  and  $g, h \in G$ , we denote the commutator of  $g$  and  $h$  by  $[g, h] := ghg^{-1}h^{-1}$ . We also denote  $g^h = hgh^{-1}$ . For two subgroups  $G_1, G_2 \leq G$ , we denote by  $[G_1, G_2]$  the subgroup of  $G$  generated by commutator elements of the form  $[g_1, g_2]$ , where  $g_1 \in G_1, g_2 \in G_2$ . We also denote  $G' := [G, G]$ .

**Definition 3.1.** *The (unrestricted) wreath product  $A \wr B$  is defined as a semidirect product  $A^B \rtimes B$ , with the multiplication  $(f_1 b_1)(f_2 b_2) = f_1 f_2^{b_1} b_1 b_2$  for all  $f_1, f_2 \in A^B, b_1, b_2 \in B$ , where  $f_2^{b_1} : B \rightarrow A$  is defined as  $f_2^{b_1}(b) = f_2(bb_1)$  for  $b \in B$ . In the definition,  $A^B$  is the set of maps from  $B$  to  $A$  with a group structure defined by coordinate-wise multiplication.*

Let

$$(4) \quad A_f := \langle a_1, a_2, \dots, b_1, b_2, \dots \mid [a_i, a_j], [a_i, b_j], [b_i, b_j], a_i^{f(i)} = b_i, i, j \in \mathbb{N} \rangle.$$

be as in the previous section. In order to construct the group  $G_f$ , first, we will describe an embedding of the group  $A_f$  into a subgroup  $L$  of the unrestricted wreath product  $A_f \wr \langle z \rangle$ , where  $\langle z \rangle$  is an infinite cyclic group generated by  $z$ . Define the functions  $f_i : \langle z \rangle \rightarrow H, i = 1, 2, \dots$ , as

$$f_i(z^n) = \begin{cases} a_i^n & \text{if } i \text{ is odd,} \\ b_i^n & \text{if } i \text{ is even} \end{cases}$$

where

$$\bar{i} = \begin{cases} (i-1)/2 & \text{if } i \text{ is odd,} \\ i/2 & \text{if } i \text{ is even.} \end{cases}$$

Note that

$$(5) \quad [z, f_i](z^n) = (zf_i z^{-1} f_i^{-1})(z^n) = (zf_i z^{-1})(z^n) f_i^{-1}(z^n) = \begin{cases} a_i^{n+1} a_i^{-n} = a_i & \text{if } i \text{ is odd,} \\ b_i^{n+1} b_i^{-n} = b_i & \text{if } i \text{ is even.} \end{cases}$$

That is,  $[z, f_i]$ , regarded as a map from  $\langle z \rangle$  to  $A_f$ , is the constant map equal to  $a_{\bar{i}}$  or  $b_{\bar{i}}$  depending on the parity of  $i$ .

Denote  $L = \langle z, f_1, f_2, \dots \rangle < A_f \wr \langle z \rangle$ . The next step is to embed the group  $L$  into a two generator subgroup  $G_f = \langle f, s \rangle$  of the full wreath product  $L \wr \langle s \rangle < A_f \wr \langle z \rangle \wr \langle s \rangle$ , where  $\langle s \rangle$  is another infinite cyclic group, generated by  $s$ . To that end, define the function  $F : \langle s \rangle \rightarrow L$  as

$$F(s^n) = \begin{cases} z & \text{if } n = 0, \\ f_n & \text{if } n > 0, \\ 1 & \text{otherwise.} \end{cases}$$

Then, for  $i \geq 0$ , we have

$$(6) \quad [F, F^{s^i}](s^n) = (F F^{s^i} F^{-1} F^{-s^i})(s^n) = \begin{cases} [z, f_i] & \text{if } n = 0, \\ 1 & \text{otherwise} \end{cases}$$

and for  $i < 0$ , we have

$$(7) \quad [F, F^{s^i}](s^n) = (F F^{s^i} F^{-1} F^{-s^i})(s^n) = \begin{cases} [f_i, z] & \text{if } n = i, \\ 1 & \text{otherwise.} \end{cases}$$

Combining (6) and (7), we get that for  $i \geq 0$ ,

$$(8) \quad [F, F^{s^i}] = [F, F^{s^{-i}}]^{-s^i}.$$

Now, define

$$G_f = \langle F, s \rangle < A_f \wr \langle z \rangle \wr \langle s \rangle.$$

Note that, by (5) and (6), it directly follows that  $A_f$  embeds into  $G_f$ . Namely, the map  $\psi : a_i \mapsto [F, F^{s^{2i+1}}], b_i \mapsto [F, F^{s^{2i}}]$  induces an embedding  $\Psi$  of  $A_f$  into  $G_f$ .

**Lemma 3.1.** *The embedding  $\Psi : A_f \hookrightarrow G_f$  is effective. Moreover, the lengths of the images of elements of  $A_f$  are being preserved under  $\Psi$  with respect to the length functions  $\ell$ .*

*Proof.* Indeed, for each  $i, n \in \mathbb{N}$ ,  $\Psi : a_i^n \mapsto [F, F^{s^{2i}}]$  and  $\Psi : b_i^n \mapsto [F, F^{s^{2i+1}}]$ . Now, note that by a straightforward checking we have  $[F, F^{s^i}]^n = [F^n, F^{s^i}]$ . Finally, for  $[F^n, F^{s^i}] \in \{F^{\pm 1}, s^{\pm 1}\}^*$ ,  $\ell_{A_f}(a_i^n) = \ell_{A_f}(b_i^n) = \log(ni) < \ell([F^n, F^{s^i}]) < 8 \log(ni)$ , which implies the statement of the lemma.  $\square$



After defining the group  $G_f$ , our next task is to show that the discrete log problem in  $G_f$  is polynomially equivalent to the computation of  $f : \mathbb{N} \rightarrow \mathbb{N}$ .

Note that by iteratively applying rewritings of the form  $s^k F^l \rightarrow (F^{s^k})^l s^k$ , every word formed by the alphabet  $\{F^{\pm 1}, s^{\pm 1}\}^*$ , in polynomial time (in fact, in linear time) can be transformed into a word of the form

$$(9) \quad w = (F^{s^{\alpha_1}})^{\beta_1} (F^{s^{\alpha_2}})^{\beta_2} \dots (F^{s^{\alpha_n}})^{\beta_n} s^\delta,$$

where  $\delta, \alpha_i, \beta_i, i = 1, \dots, n$ , are some integers. This means that every element of  $G_f$  can be decomposed as in (9).

**Lemma 3.2.** *For  $\alpha, \beta, \gamma, k, l, m \in \mathbb{Z}$ , we have  $[(F^{s^\alpha})^k, (F^{s^\beta})^l], (F^{s^\gamma})^m = 1$ .*

*Proof.* From the above discussion, for any  $n \in \mathbb{Z}$ ,  $[(F^{s^\alpha})^k, (F^{s^\beta})^l](s^n)$ , as a map from  $\langle z \rangle$  to  $G_f$  is a constant map, taking values from the commutative group  $A_f$ . Therefore,  $[(F^{s^\alpha})^k, (F^{s^\beta})^l](s^n)$  commutes with  $z$  in  $A_f \wr \langle z \rangle$ . This implies that  $[(F^{s^\alpha})^k, (F^{s^\beta})^l], (F^{s^\gamma})^m = 1$ .  $\square$

Let us denote  $\mathcal{F} = \langle F^{s^\alpha} \mid \alpha \in \mathbb{Z} \rangle < G_f$  and denote  $\mathcal{F}' = [\mathcal{F}, \mathcal{F}] < \mathcal{F}$ . As an immediate consequence of Lemma 3.2, we get

$$\mathcal{F}' = \langle ([F, F^{s^\beta}]^{s^\gamma})^l \mid \beta, \gamma, l \in \mathbb{Z} \rangle.$$

As yet another immediate corollary from Lemma 3.2, we get the following.

**Corollary 3.3.**  $\mathcal{F}' = Z(\mathcal{F})$ , that is  $\mathcal{F}'$  coincides with the center of  $\mathcal{F}$ .

**Lemma 3.4.** *Every word  $w \in \{F^{\pm 1}, s^{\pm 1}\}^*$  can be transformed in linear time into a word that represents the same element in  $G_f$  as  $w$  and such that it is of the form*

$$(10) \quad \prod_{i=1}^{t_1} (F^{s^{\alpha_i}})^{k_i} \prod_{i=1}^{t_2} ([F, F^{s^{\beta_i}}]^{s^{\gamma_i}})^{l_i} s^y,$$

where  $\alpha_1 > \alpha_2 > \dots > \alpha_{t_1}$ ,  $(\beta_1, \gamma_1) >_{lex} (\beta_2, \gamma_2) >_{lex} \dots >_{lex} (\beta_{t_2}, \gamma_{t_2})$ ,  $\beta_i > 0$ ,  $i = 1, \dots, t_2$ ,  $k_i \neq 0$  if  $t_1 \neq 0$  and  $l_i \neq 0$  if  $t_2 \neq 0$ . Moreover, the parameters  $\alpha_1, \dots, \alpha_{t_1}$ ,  $k_1, \dots, k_{t_1}$ , and  $y$  are uniquely defined for any given element of  $G_f$ .

*Proof.* Indeed, one can start with a presentation of the form (9), then iteratively apply rewritings of the form  $ab \rightarrow [a, b]ba$  in order to collect the terms  $(F^{s^{\alpha_i}})^{\beta_i}$ ,  $(F^{s^{\alpha_j}})^{\beta_j}$  for  $\alpha_i = \alpha_j$  together. Because of Lemma 3.2, the commutator terms that will occur in the process of this rewriting are of the form  $([F, F^{s^\beta}]^{s^\gamma})^l$ . Now, because of the equation (8), without loss of generality, we can always assume that  $\beta > 0$ . Also, as  $\mathcal{F}' = Z(\mathcal{F})$  due to Corollary 3.3, we can combine the terms of the form  $([F, F^{s^\beta}]^{s^\gamma})^l$  in a way to obtain the decomposition (10).

Now, let us show the uniqueness of the parameters  $\alpha_1, \dots, \alpha_{t_1}$ ,  $k_1, \dots, k_{t_1}$  and  $y$  in the decomposition (10). The uniqueness of  $y$  follows immediately from the construction (basic properties of semi-direct products) and the fact that  $\langle s \rangle$  is an infinite cyclic group. To show the uniqueness of the other parameters, let us denote  $\tilde{F} := \prod_{i=1}^{t_1} (F^{s^{\alpha_i}})^{k_i} \prod_{i=1}^{t_2} ([F, F^{s^{\beta_i}}]^{s^{\gamma_i}})^{l_i}$  and regard it as a map from  $\langle s \rangle$  to  $L = \langle z, f_1, f_2, \dots \rangle < A_f \wr \langle z \rangle$ . Then, note that, for  $i = 1, \dots, t$ ,  $\tilde{F}(s^{-\alpha_i})$  is of the form  $\tilde{f}z^{k_i}$ , where  $\tilde{f} \in \langle f_1, f_2, \dots \rangle$ ; whereas, if  $\alpha \in \mathbb{Z} \setminus \{\alpha_1, \dots, \alpha_t\}$ , then  $\tilde{F}(s^{-\alpha}) \in \langle f_1, f_2, \dots \rangle$ . Also, since  $L$  splits as  $L = \langle f_1, f_2, \dots \rangle^{\langle z \rangle} \rtimes \langle z \rangle$ , we get that in any decomposition of an element of  $G_f$  in the form (10), the parameters  $\alpha_1, \dots, \alpha_{t_1}$ ,  $k_1, \dots, k_{t_1}$  are uniquely defined as well.

□

**Corollary 3.5.** *The word problem in  $G_f$  in linear time reduces to the word problem in  $A_f$  with respect to the presentation (4). In particular, the word problem in  $G_f$ , up to a polynomial factor, can be computed in time  $T_{f^{-1}}(n)$ , where  $n$  is the length of the input with respect to  $\ell$ .*

*Proof.* Let  $g \in G_f$  be given as a word in  $\{F^{\pm 1}, s^{\pm 1}\}^*$ . By Lemma 3.4, in linear time, a canonical decomposition of the form (10) can be found for  $g$ . Again, by Lemma 3.4, if  $g = 1$ , then in its canonical decomposition the parameters  $\alpha_1, \dots, \alpha_{t_1}$ ,  $k_1, \dots, k_{t_1}$  and  $y$  would all be equal to 0. Therefore, assuming that  $\alpha_1, \dots, \alpha_{t_1}$ ,  $k_1, \dots, k_{t_1}$  and  $y$  are all equal to 0, checking whether or not  $g = 1$  would be equivalent to checking whether or not

$$(11) \quad \prod_{i=1}^{t_2} ([F, F^{s^{\beta_i}}]^{s^{\gamma_i}})^{l_i}$$

is equal to 1 in  $G_f$ , where the notation comes from (10). To check this, one can regroup the terms  $[F, F^{s^{\beta_i}}]^{s^{\gamma_i}}$  according to the values  $\gamma_i$ , that is, for each  $\gamma \in \{\gamma_1, \dots, \gamma_{t_2}\}$ , one can consider the subproduct of terms  $[F, F^{s^{\beta_i}}]^{s^{\gamma_i}}$  for which  $\gamma_i = \gamma$ . Notice that, by (6) and (5), checking whether or not (11) represents 1 in  $G_f$  would be the same as to check whether or not the element in  $A_f$  that corresponds to those subproducts via the correspondence  $[F, F^{s^{\beta_i}}]^{s^{\gamma_i}} \leftrightarrow a_{\beta_i}^{l_i} \in A_f$  (for each  $\gamma \in \{\gamma_1, \dots, \gamma_{t_2}\}$ ) represent the trivial element of  $A_f$ . Thus the word problem in  $G_f$  in linear time reduces to the word problem in  $A_f$ .

□

**3.3. The Discrete Log Problem in  $G_f$ .** In this subsection we show that the discrete log problem in  $G_f$  is polynomially equivalent to the computation of  $f$ .

**Definition 3.2.** *For any  $g \in G_f$ , a decomposition of  $g$  in the form (10), where the parameters in the form satisfy the conditions from Lemma 3.4, we call a **canonical decomposition** or **canonical form** of  $g$ .*

Let  $g, h \in G_f$ . Let us consider the equation

$$(12) \quad g^x = h, \text{ where } x \in \mathbb{Z} \text{ is the unknown.}$$

Notation: For  $g \in G_f$  that has a canonical decomposition (10), we denote  $\pi_s(g) := y$ .

Note that for  $g_1, g_2 \in G_f$ ,  $\pi_s(g_1 g_2) = \pi_s(g_1) + \pi_s(g_2)$ . Therefore, if in (12)  $\pi_s(h) \neq 0$ , then it would imply that  $x\pi_s(g) = \pi_s(h)$ , that is  $x = \pi_s(h)/\pi_s(g)$ . (In particular, it would mean that  $\pi_s(h)$  is a multiple of  $\pi_s(g)$ .) Therefore, (12) would have a solution (namely,  $x = \pi_s(h)/\pi_s(g)$ ) if and only if  $g^{\pi_s(h)/\pi_s(g)} h^{-1} =_{G_f} 1$ . Since, by Corollary 3.5, the word problem in  $G_f$  is polynomially equivalent to the computation of  $f$ , we conclude that if  $\pi_s(g) \neq 0$  or  $\pi_s(h) \neq 0$ , then solving (12) is at most as difficult as computing  $f$ . Thus we are left with the case when  $\pi_s(g) = \pi_s(h) = 0$ .

**Notation 3.1.** For  $g$  with canonical decomposition as in (10), we denote  $\deg_{\alpha_i}(g) := k_i$  for  $i = 1, 2, \dots, t_1$ . For  $\alpha \in \mathbb{N} \setminus \{\alpha_1, \dots, \alpha_{t_1}\}$ , we define  $\deg_\alpha(g) = 0$ .

Note that

$$\text{if } \pi_s(g) = 0, \text{ then } \deg_\alpha(gh) = \deg_\alpha(g) + \deg_\alpha(h) \text{ for all } \alpha \in \mathbb{N}.$$

Therefore, assuming  $\pi_s(g) = 0$ , we get  $x \deg_\alpha(g) = \deg_\alpha(h)$  for  $\alpha \in \mathbb{N}$ . This means that either  $\deg_{\alpha_i}(g) = \deg_{\alpha_i}(h) = 0$  for  $i = 1, 2, \dots, t_1$ , equivalently,  $g, h \in \mathcal{F}'$ , or there is only one possibility for  $x$ , namely  $x = \deg_{\alpha_i}(h)/\deg_{\alpha_i}(g)$ , where  $1 \leq i \leq t_1$  is such that  $\deg_{\alpha_i}(g) \neq 0$ . The last case can be dealt analogously to the above discussion, namely, it is at most as difficult as computing  $f$  to check if  $g^{\deg(h)/\deg(g)} = h$ .

Thus we are left with only one final case, which is  $g, h \in \mathcal{F}'$ .

As in the case of the word problem in  $G_f$  (see Corollary 3.5), we can regroup the factors of the form  $([F, F^{s^\beta}]^{s^\gamma})^l$ ,  $\beta > 0$ , in canonical decompositions of  $g$  and  $h$ , according to the values of  $\gamma$ . Then, for each  $\gamma$ , the corresponding product of elements of the form  $([F, F^{s^\beta}]^{s^\gamma})^l$  on the left hand side, raised to its  $x$ -th power, must be equal to the corresponding product of element on the right hand side. The crucial thing to note is that, for a fixed  $\gamma > 0$ ,

$$[F, F^{s^\beta}]^{s^\gamma} \leftrightarrow \begin{cases} a_{\bar{\beta}} & \text{if } \beta \text{ is odd} \\ b_{\bar{\beta}} & \text{if } \beta \text{ is even} \end{cases}$$

induces an isomorphism between the group  $\langle [F, F^{s^\beta}]^{s^\gamma}, \beta \in \mathbb{N} \text{ and } A_f$ , which directly follows from the equations (5) and (6), where

$$\bar{\beta} = \begin{cases} (\beta - 1)/2 & \text{if } \beta \text{ is odd,} \\ \beta/2 & \text{if } \beta \text{ is even.} \end{cases}$$

Therefore, in this case, the discrete log problem (in linear time) reduces to the discrete log problem in  $A_f$ , which, with respect to the length function  $\ell$ , is linear time equivalent to the computation of  $f : \mathbb{N} \rightarrow \mathbb{N}$  according to the Lemma 2.3.

**Conclusion 3.1.** *The discrete log problem in  $G_f$  is linear time equivalent to the computation of  $f$ .*

**Corollary 3.6** (Theorem 1.2). *There exist two-generated groups with polynomial time word problem and NP-hard discrete log problem.*

*Proof.* Indeed, let us assume that  $f : \mathbb{N} \rightarrow \mathbb{N}$  is an NP-hard function and such that  $f(n)$  serves as its polynomial-time certificate (i.e. witness), meaning that for  $(n, m) \in \mathbb{N} \times \mathbb{N}$ , one in polynomial time can verify whether or not  $f(n) = m$ . Then, according to Corollaries 3.5, 2.2, the word problem in  $G_f$  will be solvable in polynomial time, while by Conclusion 3.1, the discrete log problem in  $G_f$  is NP-hard.  $\square$

#### 4. A NEW CRYPTOGRAPHIC PROTOCOL

General idea:

We start with two functions  $f, g : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$  such that  $f$  is a one-way function. We need the following commutative relation to hold between  $f$  and  $g$ : for every  $n, p, q \in \mathbb{N}$ ,

$$(13) \quad g(f(n, p), q) = g(f(n, q), p).$$

Next, we consider three countable abelian groups with enumerated sets of generators:

$$A = \langle a_1, a_2, \dots \rangle,$$

$$B = \langle b_1, b_2, \dots \rangle,$$

$$C = \langle c_1, c_2, \dots \rangle.$$

such that  $A \leq B \leq C$ .

We define additional relations between  $A$ ,  $B$  and  $C$  as follows: for  $m, n, p, q \in \mathbb{N}$ ,

$$a_n^p = b_{f(n, p)}$$

and

$$b_m^q = c_{g(m, q)}.$$

**Remark 4.1.** *Note that in order to make sure that the above defined relators between the elements of  $A$ ,  $B$ , and  $C$  do not make generating elements trivial in the group  $A \times B \times C$ , we need to put extra conditions on the maps  $f, g : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ . For example, requiring them to be injective would work. However, it is not critically important if elements of the groups are represented by their symbolic forms as finite words formed by a generating alphabet. Although, it might become important once  $A, B, C$  are replaced by more concrete realizations, say, by numerical matrices.*

#### 4.1. A cryptographic key sharing scheme.

Public data: The functions  $f$  and  $g$  are public, as well as  $n \in \mathbb{N}$

Private key of Alice:  $p \in \mathbb{N}$

Private key of Bob:  $q \in \mathbb{N}$

The general idea of this cryptographic scheme is that Alice applies her private key  $p$  for  $a_n \mapsto a_n^p = b_{f(n,p)}$ . Then, Bob accesses  $b_{f(n,p)}$  and applies his private key  $q$  for  $b_{f(n,p)} \mapsto b_{f(n,p)}^q = c_{g(f(n,p),q)}$ . Moreover, Bob, again using his private key  $q$ , makes public the following  $a_n \mapsto a_n^q = b_{f(n,q)}$

The shared key between Alice and Bob:  $c_{g(f(n,p),q)}$  is the shared key between Alice and Bob. The reason why this key can be secretly shared lies in the Equation (13), as both  $b_{f(n,p)}$  and  $b_{f(n,q)}$  are publicly available. Moreover, since  $f$  is a one-way function, the shared key will stay private.

**4.2. Adaptation in the setting of our group theoretical construction:** Note that in our construction, we encoded the generating elements of  $\mathcal{C} := \langle a_1, a_2, \dots, b_1, b_2, \dots, c_1, c_2, \dots \rangle$  via generating elements  $F$  and  $s$  of the group  $G_{\mathcal{C}}$ , where  $G_{\mathcal{C}}$  is defined as  $G_f$  from Section 3 but with respect to  $\mathcal{A}$ . This allows us to adapt this scheme within the group-theoretical platform provided by  $G_{\mathcal{C}}$ . (Note that  $\mathcal{C}$  as a group is the same as  $C$ , but we emphasize that all elements  $a_i, b_i, c_i$  must be accounted for the definition of  $G_{\mathcal{C}}$ .)

#### 4.3. Possible candidates for $f, g : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ .

A general approach via group actions: In this approach, we will assume that  $f$  and  $g$  in fact coincide. Let  $\Gamma = \{1 = g_0, g_1, g_2, \dots\}$  be some countable group with computably enumerated elements  $1 = g_0, g_1, \dots$ . Let  $X = \{x_1, x_2, \dots\}$  be a countable set with computably enumerated elements such that  $\Gamma$  acts on  $X$  via *commutative group action*. Let  $\tilde{f} : X \rightarrow X$  be a one-way function that is a permutation of  $X$

In this setting, the value of  $f(n, p)$  can be defined as the enumeration of the element  $g_p \cdot \tilde{f}(x_n)$ . Note that, in this setting, since  $f$  and  $g$  coincide, Equation (13) turns into

$$f(f(n, p), q) = f(f(n, q), p),$$

which is equivalent to

$$(g_q \cdot g_p) \tilde{f}(x_n) = g_q \cdot (g_p \cdot \tilde{f}(x_n)) = g_q \cdot (g_p \cdot \tilde{f}(x_n)) = (g_p \cdot g_q) \tilde{f}(x_n),$$

which holds because by our assumption  $G \curvearrowright X$  is commutative.

## 5. CONCLUSIONS

We have also initiated the study of the cryptographic utility of this type of construction, touching on its post-quantum viability, and giving an example of a cryptographic scheme.

Nevertheless, our hope is that this work will spark further work investigating each of these two avenues in greater detail.

## REFERENCES

- [BBC<sup>+</sup>21] John Baena, Pierre Briaud, Daniel Cabarcas, Ray A. Perlner, Daniel Smith-Tone, and Javier A. Verbel. Improving support-minors rank attacks: applications to gemss and rainbow. *IACR Cryptol. ePrint Arch.*, page 1677, 2021.
- [Beu22] Ward Beullens. Breaking rainbow takes a weekend on a laptop. *IACR Cryptol. ePrint Arch.*, page 214, 2022.
- [BGJT14] Razvan Barbulescu, Pierrick Gaudry, Antoine Joux, and Emmanuel Thomé. A heuristic quasi-polynomial algorithm for discrete logarithm in finite fields of small characteristic. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 1–16. Springer, 2014.
- [CD22] Wouter Castryck and Thomas Decru. An efficient key recovery attack on sidh (preliminary version). Cryptology ePrint Archive, Paper 2022/975, 2022.
- [CFMR<sup>+</sup>17] Antoine Casanova, Jean-Charles Faugere, Gilles Macario-Rat, Jacques Patarin, Ludovic Perret, and Jocelyn Ryckeghem. *GEMSS : a great multivariate short signature*. PhD thesis, UPMC-Paris 6 Sorbonne Universités; INRIA Paris Research Centre, PolSys Team, 2017.
- [Dar15] Arman Darbinyan. Group embeddings with algorithmic properties. *Comm. Algebra*, 43(11):4923–4935, 2015.
- [Dar20] Arman Darbinyan. Computability, orders, and solvable groups. *J. Symb. Log.*, 85(4):1588–1598, 2020.
- [JOP14] Antoine Joux, Andrew Odlyzko, and Cécile Pierrot. The past, evolving present, and future of the discrete logarithm. *Open Problems in Mathematics and Computational Science*, pages 5–36, 2014.
- [Kup25] Greg Kuperberg. The hidden subgroup problem for infinite groups. *arXiv preprint arXiv:2507.18499*, 2025.
- [LO91] Brian A LaMacchia and Andrew M Odlyzko. Computation of discrete logarithms in prime fields. *Designs, Codes and Cryptography*, 1(1):47–62, 1991.
- [MM22] Luciano Maino and Chloe Martindale. An attack on sidh with arbitrary starting curve. Cryptology ePrint Archive, Paper 2022/1026, 2022.
- [MVO91] Alfred Menezes, Scott Vanstone, and Tatsuaki Okamoto. Reducing elliptic curve logarithms to logarithms in a finite field. In *Proceedings of the twenty-third annual ACM symposium on Theory of computing*, pages 80–89, 1991.
- [Neu60] B. H. Neumann. Embedding theorems for ordered groups. *J. London Math. Soc.*, 35:503–512, 1960.
- [Sho97] Victor Shoup. Lower bounds for discrete logarithms and related problems. In *Advances in Cryptology—EUROCRYPT’97: International Conference on the Theory and Application of Cryptographic Techniques Konstanz, Germany, May 11–15, 1997 Proceedings 16*, pages 256–266. Springer, 1997.
- [TPD21] Chengdong Tao, Albrecht Petzoldt, and Jintai Ding. Efficient key recovery for all HFE signature variants. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16-20, 2021*,

*Proceedings, Part I*, volume 12825 of *Lecture Notes in Computer Science*, pages 70–93. Springer, 2021.

SORBONNE UNIVERSITY, FRANCE

*Email address:* `christopher.battarbee@lip6.fr`

UNIVERSITY OF SOUTHAMPTON, UK

*Email address:* `A.Darbinyan@soton.ac.uk`

THE CITY UNIVERSITY OF NEW YORK, QUEENS COLLEGE, USA

*Email address:* `delaram.kahrobaei@qc.cuny.edu`